Modern Polars: A comparison of the Polars and Pandas dataframe libraries (kevinheavey.github.io)

132 points by mutant_self 9 months ago | hide | past | favorite | 62 comments

saeranv 9 months ago | next [–]

Does anyone else find the Polars syntax kind of clunky and ambiguous?

For example, from the link, here's how Polars and Pandas handles manipulating data in a subset of a dataframe:

```
f = pl.DataFrame({'a': [1,2,3,4,5], 'b':[10,20,30,40,50]})
# Polars
f.with_column(
    pl.when(pl.col("a") <= 3)
    .then(pl.col("b") // 10)
    .otherwise(pl.col("b"))
)
# Pandas
f.loc[f['a'] <= 3, "b"] = f['b'] // 10
```

Its not clear in the Polars approach that the column "b" is being modified. An additional minor nitpick here is the use of when/then/otherwise for their conditional logic. Aren't these just if/else-if/else conditions? It's seems more in line with mathematical/python convention to use if/else... am I missing something?

The Pandas equivalent, on the other hand, is much more concise, and more explicit. It also seems more mathematical to me. Polars mutates the dataframe, whereas in Pandas a function is applied to a dataframe indexed like a matrix. Pandas also benefits from it's reliance on symbolic notation, it makes everything visually clearer, whereas in Polars, the use of pl.col("b") and other similar methods contribute to multiple nested brackets and redundant naming calls contributing to less interpretability.

I know there's a lot of thought thats been put into Polars, so I assume I'm missing some of the advantages of the Polars approach, and would appreciate anyone who can shed some light on it.

I do understand, and partially agree, with the idea that indexing in Pandas leads to a lot of bugs. But in the example above, Pandas isn't really using indexing, it's using a boolean map to "index" the values from the same dataframe, so should be fairly robust. Is there a reason why Polars is trying to avoid this kind of filtering in the row/column indices?

ritchie46 9 months ago | parent | next [–]

Polars author here.

> Aren't these just if/else-if/else conditions? It's seems more in line with mathematical/python convention to use if/else... am I missing something?

Yes, they are. But if you look at pandas `f['a'] <= 3` a boolean mask is created on eagerly, on the fly. Pandas has zero chance to do anything clever here.

And yes, `when.then.otherwise` is exactly `if else`, but if `if else` is already a keyword in python so we cannot use them. `when, then, otherwise` are close synonyms.

The benefit of using the `when().then().otherwise()` expression is that it is lazy. We don't do anything until we need to materialize the result. Then the optimizer has a chance to see the query a a whole and determine if the `mask` can be reused, is not needed, should be done somewhere else, etc.

> Polars mutates the dataframe,

Almost all polars methods are pure. There will be no dataframe mutated, but a new dataframe created.

> Is there a reason why Polars is trying to avoid this kind of filtering in the row/column indices.

Yes there is. Ambiguity. I want things to be explicit. So the method names should make clear that you are selecting rows:

`df.filter`

or selecting columns:

`df.select`

or slicing

`df.slice`

In pandas this can all be done with bracket notation. I often read code something like this

`df[foo] = bar` and wondered what kind of datatype was stored into `foo`.

Indexes has the same read complexity. I often read/saw queries that showed a different outcome after a `reset_index` call. I like things to be more explicit. This may cost some keystrokes, but future me/us can more easily understand what is going on.

yowlingcat 9 months ago | root | parent | next [–]

> Yes, they are. But if you look at pandas `f['a'] <= 3` a boolean mask is created on eagerly, on the fly. Pandas has zero chance to do anything clever here.

Isn't this just an implementation detail? It seems like it wouldn't be tough to turn this into syntactic sugar rather than a forced eager evaluation. IE, `f['a'] <= 3` could just as easily evaluate into a computation graph rather than the evaluation of that graph. For example, I could imagine something like so:

```
from polars.dataframe import LazyDataFrame, DataFrame

def fn():

  ...

  ldf = LazyDataFrame(df)
  # this mutates the computation graph but doesn't evaluate
  ldf.loc[f['a'] <= 3, "b"] = f['b']
  df = DataFrame(ldf)
  return df
```

This is a toy example so I'm not sure if the part around evaluation makes complete sense, but it seems like how pandas eagerly evaluates the frame is a shortcoming of its implementation and model, rather than the syntactic sugar itself.

To be even more specific, this is the way SQLAlchemy does it. You could have something like this:

```
from models import Contact

def fn():

  ...

  # doesn't evaluate; could trivially be done as Contact[Contact.name == 'John']
  filtered_contact_exp = Contact.filter(Contact.name == 'John')
  # actually evaluates
  filtered_contacts = filtered_contact_exp.all()
  return filtered_contacts

```

And SQLAlchemy knows not to actually trigger the evaluation until you do something like `.all()`. Why not adopt this kind of pattern with Polars?

FridgeSeal 9 months ago | parent | prev | next [–]

> Does anyone else find the Polars syntax kind of clunky and ambiguous?

I've used pandas a lot, but I've come to the opposite conclusion.

In my experience, these pandas expressions end up being bracket soup, and become increasingly fragile to hold in your head while you try and figure out just which n rows and columns you're looking at.

Couple that with pandas opaqueness around copy-vs-view and the blurring of lines between API's for selection, vs API's for mutation and you get an unpleasant experience.

This particular pandas example is simpler, but it doesn't take much IME for pandas df's to end up far more unreadable.

I'll gladly take polars saner API if it means I don't have to play "data frame lisp bracket-matching" games ever again.

ollysb 9 months ago | parent | prev | next [–]

Started exploring machine learning in Python 6 months ago. Despite all the resources for learning Pandas I couldn't ever get to a point where it seemed coherent. It felt like a grab bag of tricks that accomplished various different jobs. Polars on the other hand felt really consistent and logical. Instead of having to google how to do so something in Pandas I could generally just figure out how to do it by combining the the simpler operations that Polars provides.

calpaterson 9 months ago | parent | prev | next [–]

Yeah, tried Polars a couple of times: the API seems worse than Pandas to me too. eg the decision only to support autoincrementing integer indexes seems like it would make debugging "hmmm, that answer is wrong, what exactly did I select?" bugs much more annoying. Polars docs write "blazingly fast" all over them but I doubt that is a compelling point for people using single-node dataframe libraries. It isn't for me.

Modin (https://github.com/modin-project/modin) seems more promising at this point, particularly since a migration path for standing Pandas code is highly desirable.

otsaloma 9 months ago | parent | prev | next [–]

To me it seems both Pandas and Polars sacrifice API for performance, just using different approaches to achieve that performance and thus differently bad API. There's obviously some amount of tradeoff there and no shame in tilting in scale in one direction, though it would be refreshing to be upfront and honest to users about that.

Additionally, Pandas seems an organically grown API. These days with more experience and more data frame implementations to learn from, it should be possible to do better, something I only partially see when looking at Polars.

sluijs 9 months ago | prev | next [–]

I could never get used to Pandas as a former user of R's tidyverse. The naming and syntax never really sticked with me. I find Polars' API much easier to reason about, and it definitely feels closer to dplyr than Pandas. I still miss the pipe operator though.

mutant_self 9 months ago | parent | next [–]

There’s a tidypolars package that appears to be well-maintained https://github.com/markfairbanks/tidypolars

psimm 9 months ago | parent | prev | next [–]

I feel the same. The closest to tidyverse in Python I've seen is siuba, a neat wrapper around pandas. Tidypolars is great too.

Lately, I've used DuckDB to write SQL that manipulates pandas data frames.

lvass 9 months ago | parent | prev | next [–]

There's a great polars wrapper for Elixir called Explorer if you want pipes.

bfung 9 months ago | prev | next [–]

> Many of them are academic or quant types who seem to have some complex about being “bad at coding”.

Glad I’m not the only one who’s noticed this.

Coupled with this (which leads to: [I’m bad at coding so I won’t spend effort doing it even 1/2 way good]), and pandas having the most abstraction obfuscation of underlying data types, production can become a hot flaming mess that takes months to fix and scale up even linearly w/#of customers :sweat:

stinos 9 months ago | parent | next [–]

Glad I’m not the only one who’s noticed this.

Second. I understand that because of the the places I work I encounter this more than 'standard' (say web dev), but it's painful to see how much time and money this attitude seems to cost. Anecdotal rant incming, typical example encountered multiple times: person is really good at math but subpar at programming, but just enough to make it through a PHD (though I'm like 99% sure it's impossible there were no mistakes in that code). Anyway: pretty much every meeting the "I'm bad at programming" and "I don't really know anything about language/framework/thing X" is mentioned and used as if it's a valid excuse for messing up. But the worst part is: instead of just acting on it and learning and trying to improve, there's hardly any progress and without strict guidance anything touched by said persons turns into a trainwreck in no time. Again anecdotal, but I see this much less often with engineers.

brahbrah 9 months ago | root | parent | next [–]

I have the same dynamic at my job. It’s a classic case of they can’t do what I can do and I can’t do what they can so let’s work together. It’s painful but necessary. I feel like it’s a perfectly valid excuse though. They have training in some other concepts that make them valuable, we can’t expect people to know what they know and learn what we know too. It’s why we work in teams. Although the people who are highly skilled in the analytical and engineering disciplines are worth their weight in gold.

erikcw 9 months ago | prev | next [–]

I just tried polars for this first time this week. I ported a data pipeline from pandas and I was blown away by the performance yield. Function went from a 60 min runtime with pandas to ~1:30 in polars!

I’ve been using pandas for years and had no issues picking up the syntax. Can’t recommend giving it a try enough.

brahbrah 9 months ago | parent | next [–]

By any chance were you iterating over your pandas dataframe or using .apply? I’d be surprised by any properly formatted (i.e. vectorized) pandas operation that takes that long for data that fits in memory

mutant_self 9 months ago | root | parent | next [–]

Here's an example of idiomatic Pandas taking 10 minutes while Polars takes 7 seconds: https://www.pola.rs/posts/the-expressions-api-in-polars-is-a...

brahbrah 9 months ago | root | parent | next [–]

I'm not saying that polars isn't faster. In fact in my other comment here I mention that polars is much better than pandas at what polars does (it's not a drop in replacement). I'm just saying that most of the times (not always, and in fact in those cases we've used polars to speed it up) that I've seen painfully slow pandas operations has been due to poorly formatted pandas code.

peepeepoopoo3 9 months ago | prev | next [–]

I hate pandas with a burning passion, but one thing it does have going for it is (some) interoperability with numpy, which opens up the rest of the scipy ecosystem. How easy is it to get numpy arrays into and out of polars?

ritchie46 9 months ago | parent | next [–]

Very easy.

`pl.from_numpy` and `series.to_numpy` are your friend here. For 1D columns, we often can be zero copy as well.

Besides that we support numpy ufuncs for `Series` and `Expressions`. As OP pointed out:

https://kevinheavey.github.io/modern-polars/performance.html...

Numpy can be used to speed up some functions by utilizing numpy ufuncs. Numpy drops the GIL and therefore they can still be executed in parallel.

Hasnep 9 months ago | parent | prev | next [–]

An alternative I found recently is RedFrames [1] which wraps Pandas dataframes in a more consistent interface. That might be a better alternative if you need easy compatibility with Pandas.

[1] https://github.com/maxhumber/redframes

fbdab103 9 months ago | root | parent | next [–]

Though that does look slick, the project is only ~5 months old. Which is a bit young for me to jump aboard.

yuuuxt 9 months ago | root | parent | prev | next [–]

Seems RedFrames is similar to pyjanitor, which is maturer if only comparing existence time: https://github.com/pyjanitor-devs/pyjanitor

atoav 9 months ago | root | parent | prev | next [–]

Oh that looks interesting

mbernstein 9 months ago | parent | prev | next [–]

As simple as a call foo.to_numpy() it looks like.

culi 9 months ago | parent | prev | next [–]

what do you hate about pandas so much? I miss it dearly now that I don't use Python anymore

Hasnep 9 months ago | root | parent | next [–]

I'm not GP, but I find the pandas API incredibly inconsistent and difficult to remember how to do simple transformations. For example, it sometimes overloads operators because it doesn't use built in language features like lambdas. There are reasons for the inconsistency, but using the alternatives like R's tidyverse or Julia's DataFramess.jl is like night and day for me.

I found RedFrames [1] recently which wraps Pandas dataframes with a more consistent interface, it's probably what I'd use if I had to write data transformations that had to be compatible with Pandas.

[1] https://github.com/maxhumber/redframes

Galanwe 9 months ago | root | parent | prev | next [–]

Pandas gets the job done, and is overall easy to use and intuitive.

The problem is that it's a huge pile of hacks, exceptions, anti patterns, and regressions.

The API is inconsistent, loose, full of obscure options added as quickfixes.

pupperino 9 months ago | root | parent | prev | next [–]

It really can't be said enough how pandas is a mess. It has way too much surface area and no common thread pulling it all together. This gets obvious when you work with better dataframe libs like dplyr [1] or DataFramesMeta [2]. I've worked on production systems with all of these libs, this is not gratuitous bashing.

[1] https://dplyr.tidyverse.org/ [2] https://juliadata.github.io/DataFramesMeta.jl/stable/

topaz0 9 months ago | parent | prev | next [–]

Funny seeing you here

antman 9 months ago | prev | next [–]

If I understand correctly the currently promoted libraries for dataframes are:

1. Polars if data fits in ram

2. Vaex if data do not fit in ram

3. Spark with the dataframe api (koalas) if data do not fit in a computer

Polars is great and delivers as promised

IanOzsvald 9 months ago | parent | next [–]

I'd argue a little differently. I'm co-author of O'Reilly's High Performance Python book and I've been teaching a course around this for years, often to quants.

1. Pandas if you stay in RAM, if the team and org already know this, but learn about reduced-ram types (eg float32 rather than float64, categorical for strings and dt if low cardinality, new Arrow strings in place of default Object str). Pandas 1.5 has an experimental copy-on-write option for more predictable (but probably still not "predictable") memory usage, try to use a subset of team-agreed functions (eg merge over join) due to varied defaults that'll confuse colleagues (eg inner Vs left and other differences). Buying more ram is normally a cheap (if inelegant) fix.

2. Dask as it is an easy transition from Pandas (and it scales numpy math, arbitrary python non-math functions and lots more), lots of cloud scaling options too. Stays within Python ecosystem for reduced cognitive load. It is probably less resource efficient than Vaex/Polars

3. Ignore Dask and stick with Spark if your team already uses it, as it'll scale to larger workloads and you've taken the cognitive and engineering hit (pragmatism over purity)

Vaex and Polars are definitely interesting (hi Ritchie!), and great if you're doing research and are comfortable with potentially changing APIs but you have no legacy systems to worry about. You might buy yourself a lot of future manoeuvring room. You'll find fewer clues to tricky problems in SO than for Pandas, and have a harder time hiring experienced help.

ritchie46 9 months ago | root | parent | next [–]

Hi Ian ;),

It depends on what let determine the order. Hiring experience and available content, I wholeheartedly agree with your list.

But if we order by performance/memory efficiency, A single threaded, (eager), library simply will be no comparison and should not top that list. In every TPCH query we ran, polars is orders of magnitudes faster than pandas.

https://www.pola.rs/benchmarks.html

Interopability with legacy systems should not be a concern. Polars is backed by arrow memory and arrow is becoming the default data transformation layer. Other than that, you can easily convert to pandas or numpy. That single copy is often no comparison with the time lost in a pandas join. Polars and pandas can work hand in hand, you don't have to fully replace one.

It is 2023, polars is used in production and is here to stay. IMO it should seriously be considered if performance and consistency is important to you.

IanOzsvald 9 months ago | root | parent | next [–]

Hey Ritchie. Re legacy I'm thinking about wider teams in large organisations (eg SWEng system support teams) and IT mandating library upgrade frequency - switching to new libraries can have widespread impacts and the cost can be high. Polars (and Vaex) are definitely here to stay, but I think integration to existing teams may take a while. I followed the PRs around numpy data sharing but I wasn't sure on the end result. Is the data sharing copy-free (always?)? I wasn't sure what the impact was if Rust and NumPy are utilising the same bytes (or even if that was possible). Can you share some detail? Edit - reading the updated thread I your reply https://news.ycombinator.com/item?id=34298023 which says "1D often no copy", can you add any colour to when a 1D no copy can't happen and whether 2D no copy is an option?

fbdab103 9 months ago | root | parent | prev | next [–]

>But if we order by performance/memory efficiency

Right there is the disagreement. Like many (most?) people, all of my data munging is in small/medium data where 10 million+ rows is rare. A multiple of pandas performance will not be noticed for the majority of my operations.

Transitioning to a new api on performance alone is not enough to sway me. After all, I write in Python ;). If I were concerned about better throughput, my first alternative would be Dask - it should give better local performance, but could theoretically scale to enormous data without any code changes.

lr1970 9 months ago | root | parent | prev | next [–]

> In every TPCH query we ran, polars is orders of magnitudes faster than pandas.

I have no doubts that polars is faster than pandas. But the published TPCH results [0] are fairly outdated based on polars-0.13.51 while the current polars is 0.15.13. Are there any plans to refresh the benchmarks?

[0] https://www.pola.rs/benchmarks.html

pletnes 9 months ago | parent | prev | next [–]

I actually thought polars' lazy api would allow for out-of-ram computation?

Also dask is more flexible than spark, since it lets you deal with numpy arrays and arbitrary objects better than spark can.

ritchie46 9 months ago | root | parent | next [–]

It does. Though the functionality is quite new, we will extend this.

Calling `collect(streaming=True)` on a `LazyFrame` will allow you to process datasets that don't fit into memory. This currently works for groupbys, joins, many functions, filter etc.

We will extend this to sorts and likely other operations as well.

glogla 9 months ago | root | parent | next [–]

I'm curious if you could use this not for data science tasks but for data engineering tasks - say read a csv or pull a table from oracle and store it as delta lake table or something.

I know its a boring use case, but the challenge with it is that it is a complete waste of money and carbon footprint to use Spark to process a 20 MB CSV or table with few thousand records, but tools like Pandas fall apart when you hit a 50 GB CSV or table with few billion records.

Something more efficient (say, in Rust and not Python or Java) and yet scalable (due to not fitting everything into memory) would be a great help here.

ritchie46 9 months ago | root | parent | next [–]

This is exactly what we are aiming for. There are already a lot of queries that can be processed with 100s GBS of data on my 16GB laptop.

And we will extend functionality for out of core processing. A single node can do a lot!

mooncitizen 9 months ago | root | parent | prev | next [–]

Can you please put an example about dask being more flexible than spark?

IanOzsvald 9 months ago | root | parent | next [–]

This https://docs.dask.org/en/stable/spark.html notes "However, Dask is able to easily represent far more complex algorithms and expose the creation of these algorithms to normal users [compared to spark]" linking to: http://matthewrocklin.com/blog/work/2015/06/26/Complex-Graph...

pletnes 9 months ago | root | parent | prev | next [–]

Yes, say you've got imaging data in 3 (or higher) dimensions as numpy arrays and want to run some sort of algorithm on multiple cores/machines. Could be both for data analytics and simulations.

dask.bag has generic parallel processing capabilities. Query a database, a rest api, something. Then merge into dataframes across dask workers.

prds_lost 9 months ago | parent | prev | next [–]

My understanding is polars will stream if data does not fit in RAM.

Between Polars and Spark Dataframe APIs (not Koalas) as well as the occasional dplyr, I will gladly abandon Pandas.

adgjlsfhk1 9 months ago | parent | prev | next [–]

imo this is wrong in a few ways. firstly your data fits in a computer. you can get a computer with a petabyte of storage if you need to, and spark is slow enough that doing it in a single computer will probably be faster. also while a computer with 1pb of storage is expensive, it's less expensive than splitting your data up in terms of hardware, maintenance, and software dev time costs.

secondly, your data probably fits in RAM if you actually try. your can get a computer with 60TB of RAM which is an awful lot of data.

cjbgkagh 9 months ago | root | parent | next [–]

But does the data fit in compute? Assuming computation needs to be done on the data you could quickly run out of compute power on a single machine. Especially since cost of CPU power is super linear with some pretty hard limits.

adgjlsfhk1 9 months ago | root | parent | next [–]

You can get 256 cores of Zen4 (with 8 GPUs) in a box. That's a lot of compute. You definitely can be compute constrained but if you are writing efficient code you can do a lot.

cjbgkagh 9 months ago | root | parent | next [–]

That's $44K in CPUs alone, and these clock slower than desktop cores so it's not exactly equivalent. There are many tasks for which GPUs are not relevant. I have an expensive compute limited task and for my purposes a mini cluster of desktop CPUs still seems to be the way to go.

adgjlsfhk1 9 months ago | root | parent | next [–]

Lets say that the cluster of desktop CPUs is 10 CPUs with 8 cores each for 80 cores at 4ghz in total. Instead you can get a server with 64 cores at 2.5 GHz and 1TB of memory for $50k from Dell (adding another 64 cores would be about $6000 more). While this may sound like a lot less performance, you won't be wasting a ton of performance reading data from disk and communicating over a network All of your cores share the same RAM and cache so your computation will be a ton faster. Furthermore, your cores will all be running the same instruction set so you can take more advantage of vectorization). If you aren't using compute frequently enough to justify a server like this, you can get that level of performance and memory from AWS for $4 an hour.

cjbgkagh 9 months ago | root | parent | next [–]

My task has next to no network or hdd. Data parallel and the task is mostly branches, so GPUs are not effective either. And why use 8 core desktops when 16 core desktops exist. 10 desktops at $1.5K each is way cheaper. It's also the kind of data that can go into the cloud. Not saying it's the typical workload but it is my workload. Also it's CPU bound so they go 100% all the time.

cjbgkagh 9 months ago | root | parent | next [–]

Edit: cannot go into the cloud…

jacquesm 9 months ago | root | parent | prev | next [–]

And if your workload is FP and nicely parallelizes and fits in 12 GB or less than you might be able to get that kind of performance for the price of a high end GPU + a box to stick it in. GPUs are insanely cost effective for problems that match their geometry. So much so that it can be worth thinking about it for a little while to see if you can make it fit.

brahbrah 9 months ago | prev | next [–]

I like polars a lot. It's better than pandas at what it does. But it only accounts for a subset of functionality that pandas does. Indexes are not just some implementation details of dataframes. They are fundamental to the representation of data in a way where dimensional structure is relevant. Polars is great for cases where you want to work with data in "long" format, but that's not always the most convenient way to work with data. Let's say you want to get the difference in 15 day ahead temperature forecasts between forecasts on 2 different mark dates, for the forecast days they overlap (say the data consists of forecasted date, country, state equivalent, temp). In long format (necessarily in polars, optionally in pandas) you have to

do:

```
    Merge df 1 and 2 on country, state and forecasted date, then create a new column of the diff between the 2
temp columns, then drop the 2 original temp columns.
```

In a format where your indexes are forecasted dates on the rows and multiindex of country, state on the columns, you just have to do:

```
    df1 - df2
```

The way I see pandas is a toolkit that lets you easily convert between these 2 representations of data. You could argue that polars is better than pandas for working with data in long format, and that a library like xarray is better than pandas for working with data in the dimensionally relevant structure, but there is a lot of value in having both paradigms in one library with a unified api/ecosystem.

bobbylarrybobby 9 months ago | prev | next [–]

A bit off topic but I would love to see the conciseness of the python polars API make it into rust. Mapping custom functions over a series is incredibly painful.

porker 9 months ago | prev | next [–]

> f.loc[f['a'] <= 3, "b"] = f['b']

Why isn't that saying to assign the value of column b to these locations? Reading the code (and not being a Pandas user) I expected it to be

f.loc[f['a'] <= 3, "b"] = f['a']

Also the "// 10" comment is most confusing as looking at the result it matches 10, 20 & 30 in column b and replaces them with the matching values from column a

149764 9 months ago | parent | next [–]

In python // is integer division, comments start with #.

porker 9 months ago | root | parent | next [–]

Boy I feel stupid. Thanks for setting me straight.

Phlogi 9 months ago | prev | next [–]

Is there a way to download the whole book as a ebpub or kindle compatible document?

__marvin_the__ 9 months ago | parent | next [–]

Not the author but it seems that the site was made using Quarto [1] which uses pandoc [2] behind the scenes for producing the final output. The pandoc website suggests EPUB is possible.

[1] https://quarto.org/docs/get-started/authoring/text-editor.ht...

[2] https://pandoc.org/

mutant_self 9 months ago | root | parent | next [–]

Author here: you are correct but the EPUB and PDF stuff actually didn’t work for this (it exited while rendering). iirc one of the problems was Quarto didn’t know how to format Polars dataframes

blakeburch 9 months ago | prev | next [–]

Really appreciate this side-by-side guide. Didn't realize Polars could still be used with Python, and the speed improvements seem to be drastic.

May need to scope if it's worth updating our open-source connectors.

n8henrie 9 months ago | prev [–]

I've been considering writing something like this for the last several days, glad the author has taken this off my plate :)